

Regd.No:23b01a1204

EXPERIMENT-3

DATE:23-12-2025

In [34]:

```
# k nearest neighbor using Euclidean distance

import math
def euclidean_distance(a, b):
    dist=0
    for i in range(len(a)):
        if a[i] is not None and b[i] is not None:
            dist+=(a[i]-b[i])**2
    return math.sqrt(dist)

def nearest_neighbor(data,incomplete):
    miss_idx=incomplete.index(None)
    nearest=None
    min_dist=float("inf")
    for vector in data:
        d=euclidean_distance(incomplete, vector)
        if d<min_dist:
            min_dist=d
            nearest=vector
    result=list(incomplete)
    result[miss_idx]=nearest[miss_idx]
    print("Nearest neighbor:",nearest)
    print("Imputed value:",nearest[miss_idx])
    print("Final vector:",tuple(result))

n=int(input("No.of complete vectors: "))
data=[]
for i in range(n):
    data.append(tuple(map(int,input(f"Vector {i+1}: ").split()))))
temp=input("Incomplete vector (None): ").split()
incomplete=tuple(None if x=="None" else int(x) for x in temp)
nearest_neighbor(data,incomplete)
```

No.of complete vectors: 3

Vector 1: 1 1 1

Vector 2: 1 1 2

Vector 3: 1 1 3

Incomplete vector (None): 1 None 2

Nearest neighbor: (1, 1, 2)

Imputed value: 1

Final vector: (1, 1, 2)

In [30]:

```
# Write a Python program that takes a matrix as input, collects elements #column-wise,
identifies missing values in each column,computes the avera #of available values in that
column,and replaces missing values with comp

def compute_avg(r,c,matrix):
    avg=[]
    for j in range(c):
        s=0
        cnt=0
        for i in range(r):
```

```

        if matrix[i][j] is not None:
            s+=matrix[i][j]
            cnt+=1
        avg.append(s/cnt)
    for i in range(r):
        for j in range(c):
            if matrix[i][j] is None:
                matrix[i][j]=avg[j]
    print("UpdatedMatrix:")
    for row in matrix:
        print(row)

r=int(input("No.ofRows: "))
c=int(input("No.ofColumns: "))
matrix=[]
matrix=[[int(x) if x != "None" else None
          for x in input(f"Enter row {i+1} values (or None): ").split()]
         for i in range(r)]
compute_avg(r,c,matrix)

```

No.of Rows: 4
 No.of Columns: 3
 Enter row 1 values (or None): 10 20 None
 Enter row 2 values (or None): 15 None 30
 Enter row 3 values (or None): None 25 35
 Enter row 4 values (or None): 20 30 40
 Updated Matrix:
 [10, 20, 35.0]
 [15, 25.0, 30]
 [15.0, 25, 35]
 [20, 30, 40]

In [28]:

```

# Write a Python program to apply range-based scaling to a given set
# of numerical values

def scaled_values(values):
    min_val=min(values)   max_val=max(values)   range_val=max_val-min_val;
    scaled_val=[round((val-min_val)/range_val,2) for val in values] print("scaled
    values: ",scaled_val)

values=list(map(int,input("Enter values: ").split()))
scaled_values(values)

```

Enter values: 12 20 28 35 40
 scaled values: [0.0, 0.29, 0.57, 0.82, 1.0]

In [27]:

```

# Write a Python program to normalize a given set of numerical values usi
# the standardization process
def standardized_values(values):
    mean_val=sum(values)/len(values)
    std_dev=(sum((x-mean_val)**2 for x in values)/len(values))**0.5
    standardized=[round((x-mean_val)/std_dev,2) for x in values]
    print("standardized values: ",standardized)

values=list(map(int,input("Enter values: ").split()))
standardized_values(values)

```

Enter values: 10 20 30
 standardized values: [-1.22, 0.0, 1.22]

